

دانشگاه تهران

پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر

تمرین شماره: ۶
شبکه‌های عصبی و یادگیری عمیق

نام و نام خانوادگی: آرتین توسلی

شماره دانشجویی: ۸۱۰۱۰۲۵۴۳

نیم‌سال دوم
سال تحصیلی ۴۰۴-۴۰۵

فهرست مطالب

۳	Naive GAN + WGAN-GP + BEGAN	۱
۳	تئوری	۱.۱
۳	مدل Naive GAN	۱.۱.۱
۵	مدل WGAN-GP	۲.۱.۱
۷	مدل BEGAN	۳.۱.۱
۹	معیار ارزیابی FID	۴.۱.۱
۱۰	پیاده سازی	۲.۱
۱۰	مدل Naive GAN	۱.۲.۱
۱۲	مدل WGAN-GP	۲.۲.۱
۱۵	مدل BEGAN	۳.۲.۱
۲۱	مقایسه مدل های GAN	۴.۲.۱

فهرست تصاویر

۱۱	نمودار خام تابع هزینه مولد و متمایزگر در طول آموزش (به ازای هر Batch)
۱۲	نمودار میانگین تابع هزینه مولد و متمایزگر به ازای هر Epoch
۱۳	نمونه ارقام تولید شده توسط شبکه مولد در پایان اپیاک 30
۱۵	نمودار خام تابع هزینه مولد و منتقد در طول آموزش WGAN-GP (به ازای هر Batch)
۱۶	نمودار میانگین تابع هزینه مولد و منتقد به ازای هر Epoch (WGAN-GP)
۱۷	نمونه ارقام تولید شده توسط شبکه مولد WGAN-GP در پایان اپیاک 30
۱۸	نمودار میانگین تابع خطای بازسازی مولد و متمایزگر به ازای هر Epoch (BEGAN)
۱۹	نمودار معیار همگرایی سراسری (M) و متغیر کنترل تعادل (k) در طول آموزش
۲۰	نمونه ارقام تولید شده توسط شبکه مولد BEGAN در پایان اپیاک 30

سوال ۱

Naive GAN + WGAN-GP + BEGAN

۱.۱ تئوری

۱.۱.۱ مدل Naive GAN

این مدل از دو شبکه عصبی کاملاً جدا از هم تشکیل شده است (در نسخه های اولیه اش MLP بودند و بعداً در مثلاً DCGAN کانولوشنی شدند).

شبکه مولد (G - Generator): هدف این شبکه، یادگیری توزیع داده های واقعی (p_{data}) است تا بتواند داده های فیک تولید کند که از داده های واقعی غیرقابل تشخیص باشند. یک بردار نویز تصادفی z که از یک توزیع ساده (p_z) مانند توزیع نرمال $\mathcal{N}(0, I)$ یا یکنواخت نمونه برداری می شود و به عنوان ورودی به این شبکه داده میشود و داده فیک $\tilde{x} = G(z)$ که هم بعد با داده های واقعی است را دریافت میکنیم (مثلاً یک عکس فیکی که تولید کردیم).

شبکه متمایزگر (D - Discriminator): هدف آن تشخیص این است که یک نمونه ورودی، از توزیع داده های واقعی آمده است یا توسط Generator تولید شده است یعنی یک binary classifier که تشخیص بدهد فیک هست یا نه. نمونه x که می تواند داده واقعی ($x \sim p_{data}$) یا داده جعلی ($x = G(z)$) باشد را به عنوان ورودی میگیرد و یک عدد اسکالر در بازه $[0, 1]$ که نشان دهنده احتمال واقعی بودن داده است را خروجی میدهد. (با استفاده از تابع فعال ساز Sigmoid در لایه آخر).

تابع خطای این مدل در اصل یک بازی Minimax بین G و D میباشد.

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

ترم اول $E_{x \sim p_{data}(x)} [\log D(x)]$: امید ریاضی روی داده های واقعی است. شبکه D می خواهد $D(x)$ برای داده های واقعی به 1 نزدیک شود (پس لگاریتم آن به 0 میل کند تا بیشینه شود). شبکه G روی این ترم هیچ کنترلی ندارد.

ترم دوم $E_{z \sim p_z(z)}[\log(1 - D(G(z)))]$: امید ریاضی روی داده‌های فیک است. شبکه D می‌خواهد $D(G(z))$ را به 0 نزدیک کند (تا لگاریتم $1 - 0$ برابر 0 شود و بیشینه گردد). شبکه G برعکس، می‌خواهد $D(G(z))$ را به 1 نزدیک کند تا این ترم کمینه (منفی بی‌نهایت) شود.

در عمل به جای اینکه G تلاش کند $\log(1 - D(G(z)))$ را کمینه کند، طوری آموزش داده می‌شود که $\log D(G(z))$ را بیشینه کند (چون در ابتدای آموزش، شبکه G خیلی ضعیف است و D به راحتی تفاوت را می‌فهمد $D(G(z)) \rightarrow 0$ و vanishing gradient رخ می‌دهد. اثبات رو زیر کردم)

اثباتش سادس. آخرین لایه D گفتیم sigmoid دارد یعنی $\sigma(a) = D(G(z))$

حال در آموزش G داریم:

$$\nabla_{\theta} V(G, D) = \nabla_{\theta} E_{z \sim p_z(z)}[\log(1 - \sigma(a))]$$

با توجه به اینکه میدانیم مشتق سیگموئید میشود

$$(1 - \sigma(a)) * \sigma(a)$$

$$\nabla_{\theta} V(G, D) = \nabla_{\theta} E_{z \sim p_z(z)}[\log(1 - \sigma(a))] = \frac{-\nabla_a \sigma(a)}{1 - \sigma(a)} = -\sigma(a) = -D(G(z))$$

و چون طبق استدلال بالا $D(G(z)) \rightarrow 0$ پس به راحتی vanishing gradient را دیدیم.

آموزش GAN به صورت تناوبی انجام می‌شود. یعنی یک بار D را ثابت نگه می‌داریم و G را آپدیت می‌کنیم و بار دیگر برعکس. برای تعداد مشخصی از Epochها حلقه‌ی زیر را تکرار می‌کنیم:

آموزش Discriminator:

- یک مینی‌بچ شامل m نمونه از داده‌های واقعی استخراج می‌کنیم: $x^{(1)}, \dots, x^{(m)} \sim p_{data}(x)$
- یک مینی‌بچ شامل m بردار نویز تصادفی تولید می‌کنیم: $z^{(1)}, \dots, z^{(m)} \sim p_z(z)$
- وزن‌های متمایزگر (θ_d) را با استفاده از الگوریتم Gradient Ascent آپدیت می‌کنیم:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m [\log D(x^{(i)}) + \log(1 - D(G(z^{(i)})))]$$

آموزش Generator:

- یک مینی‌بچ جدید شامل m بردار نویز تصادفی تولید می‌کنیم: $z^{(1)}, \dots, z^{(m)} \sim p_z(z)$
- وزن‌های مولد (θ_g) را با استفاده از الگوریتم Gradient Ascent روی فرمول تغییر داده شده (میتوانید Gradient Descent هم بزنید بر فرمول

اصلی ولی احتمالاً نتیجه بدتری بگیرین و آموزش هم کمی سختتر بشه.)

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(D(G(z^{(i)})))$$

مدل Naive معمولاً دوست دارد Mode Collapse بشود یعنی تنوع داده های تولیدی افت پیدا کند و بخاطر این هست که چند نمونه خاص D را گول میزند و G هم میفهمد با کمک همین تابع هزینه اش کاهش پیدا میکند پس اسپم میکند همین عکس ها رو و دیگر فقط همین را تولید میکند.

۲.۱.۱ مدل WGAN-GP

در Naive GAN، تابع خطای D به گونه ای معادل با کمینه سازی Jensen-Shannon Divergence بین توزیع داده های واقعی (p_{data}) و توزیع داده های تولیدی (p_g) است. وقتی این دو توزیع با هم Overlap نداشته باشند (که در ابعاد بالا و در ابتدای آموزش تقریباً همیشه همینطور است)، فاصله JSD یک مقدار ثابت می شود و گرادیان آن صفر میشه و یعنی Vanishing Gradient داریم و گفتیم Mode Collapse هم که داریم.

در بالاتر توضیح دادم که چرا به صورت عملی تابع خطای G را عوض میکنیم تا مشکل Vanishing Gradient نخوریم. اون موقع دیگر در جهت کمینه کردن JSD حرکت نمیکنیم ولی هنوز مشکلات بزرگی داریم. Mode Collapse که هنوز هست و در مورد گرادیان هم:

$$L_G = E_{z \sim p_z} [-\log D(G_\theta(z))]$$

$$\nabla_{\theta} L_G = E_{z \sim p_z} \left[\frac{-1}{D(G_\theta(z))} \nabla_{\theta} D(G_\theta(z)) \right]$$

وقتی $0 \rightarrow D(G_\theta(z))$ که گفتیم اولای آموزش اینطوری میشه طبق فرمول بالا باعث میشه Exploding Gradient رخ بدهد یعنی این راه حل ساده ای که گفته بودیم باعث ناپایداری شدید و آموزش خیلی سخت میشه و یک ایده زده شد که بیایم یک فاصله معنادار تعریف کنیم براش بجای اینکار ها.

مدل WGAN به جای JSD، استفاده از Wasserstein-1 Distance را پیشنهاد داد. این فاصله، کمترین هزینه لازم برای تبدیل توزیع p_g به p_{data} را محاسبه می کند و مزیت بزرگ آن این است که حتی وقتی توزیع ها هیچ هم پوشانی ندارند، تابعی پیوسته و مشتق پذیر است و گرادیان های معناداری تولید می کند.

$$W(p_{data}, p_g) = \sup_{\|f\|_L \leq 1} E_{x \sim p_{data}} [f(x)] - E_{x \sim p_g} [f(x)]$$

در اینجا f شبکه ماست که باید یک-لیپشیتز باشد (یعنی اندازه گرادیان تابع نسبت به ورودی اش در هیچ نقطه ای از 1 بیشتر نشود). به دلیل اینکه خروجی شبکه دیگر احتمال (بین 0 و 1) نیست، به آن به جای Discriminator، به آن Critic می گویند.

در نسخه اولیه WGAN، برای اعمال شرط لپشیتز وزن ها را در یک بازه کوچک برش میزد ولی این ظرفیت شبکه را کاهش میدهد و در این مقاله یک ترم Regularization معرفی کرد.

تغییرات مهم

- حذف لایه Sigmoid در شبکه Critic - C: شبکه C دیگر یک Binary Classifier نیست که بگوید عکس واقعی است یا فیک. بلکه یک تخمین گر فاصله (اسکالر واقعی) است که به عکس های واقعی نمره بالاتر و به عکس های فیک نمره پایین تر می دهد. بنابراین لایه آخر نباید تابع فعال ساز Sigmoid داشته باشد.
- در WGAN-GP برای اعمال جریمه گرادیان، ما به محاسبه گرادیان شبکه به ازای هر نمونه ورودی به صورت مستقل نیاز داریم پس نباید از لایه Batch Normalization که وابستگی بین نمونه های یک Mini-batch ایجاد میکند استفاده کنیم و میتوانیم مثلاً از Instance Norm یا Layer Norm استفاده کرد برای معماری Critic (در Generator مهم نیست Batch Norm بزنیم).
- مدل WGAN-GP شرط Lipschitz-1 بودن را با اضافه کردن یک ترم جریمه به تابع خطای Critic اعمال می کند. می دانیم که یک تابع مشتق پذیر Lipschitz-1 است اگر و تنها اگر Norm گرادیان آن در همه جا کوچکتر یا مساوی 1 باشد پس خیلی راحت در تابع خطا گرادیان را جریمه میکنیم. تابع خطای Critic (L_C) به این شکل در می آید:

$$L_C = \underbrace{E_{\tilde{x} \sim p_g}[C(\tilde{x})] - E_{x \sim p_{data}}[C(x)]}_{\text{Loss WGAN Original}} + \underbrace{\lambda E_{\tilde{x} \sim p_{\tilde{x}}}[(\|\nabla_{\tilde{x}} C(\tilde{x})\|_2 - 1)^2]}_{\text{(GP) Penalty Gradient}}$$

- ترم اول و دوم (Wasserstein Loss): Critic سعی می کند فاصله بین نمره داده های واقعی $C(x)$ و داده های فیک $C(\tilde{x})$ را بیشینه کند.
- ترم سوم (Gradient Penalty): ضریب λ وزن جریمه است. ما نمی توانیم جریمه گرادیان را روی تمام فضای حالت محاسبه کنیم، بنابراین نمونه های $\tilde{x}$ را به صورت Linear Interpolation بین داده های واقعی و فیک تولید می کنیم:

$$\hat{x} = \epsilon x + (1 - \epsilon)\tilde{x}$$

که در آن ϵ یک عدد تصادفی از توزیع یکنواخت $U[0, 1]$ است. سپس گرادیان خروجی Critic نسبت به این $\hat{x}$ را حساب کرده $(\|\nabla_{\hat{x}} C(\hat{x})\|_2)$ و اگر این اندازه از 1 فاصله بگیرد، آن را با توان 2 جریمه می کنیم.

تابع خطای شبکه Generator:

$$L_G = -E_{z \sim p_z}[C(G(z))]$$

صرفاً تلاش می کند خروجی اش توسط Critic نمره بالاتری دریافت کند.

- در WGAN، شبکه C باید تا حد ممکن بهینه تر از G باشد تا بتواند فاصله واسراشتاین را به درستی تخمین بزند. پس به ازای هر یک بار آپدیت وزن

های G، شبکه C باید n_{critic} بار (معمولاً 5 بار) آموزش داده شود و وزن‌هایش آپدیت گردد.

خلاصه با جایگزینی تابع Sigmoid و JSD، با فاصله واسراشتاین، حتی زمانی که Generator عکس‌های بسیار ضعیفی تولید می‌کند و Critic به راحتی تفاوت را می‌فهمد، همچنان گرادین‌های خطی و سالمی برای یادگیری به Generator ارسال می‌شود. در Naive GAN مقدار تابع هزینه D و G هیچ معنای شهودی از کیفیت عکس‌ها به ما نمی‌داد. اما در WGAN-GP، مقدار Loss شبکه Critic به شدت با کیفیت عکس‌های تولید شده Correlation دارد. هرچه Loss اش کمتر شود، یعنی توزیع‌ها به هم نزدیکتر شده‌اند و کیفیت خروجی بهتر است. به دلیل ذات فاصله واسراشتاین که توزیع‌ها را مجبور می‌کند به طور کلی روی هم منطبق شوند (برخلاف JSD، که اگر بخشی از توزیع روی بخش کوچکی از هدف منطبق می‌شد گول می‌خورد)، تنوع عکس‌های تولیدی به شدت افزایش یافت و مدل دیگر روی تولید چند عکس خاص گیر نمی‌کند و mode collapse کاهش می‌یابد و WGAN-GP نسبت به WGAN پایداری آموزشی بیشتری دارد چون از Weight Clipping استفاده نمی‌کند.

۳.۱.۱ مدل BEGAN

در Naive GAN و WGAN ما به دنبال این بودیم که توزیع p_g را به p_{data} نزدیک کنیم. اما BEGAN به جای توزیع خود داده‌ها، با خطای بازسازی کار دارد.

تغییر اساسی در شبکه Discriminator که در Naive GAN یک Binary Classifier بود و شبکه Critic که یک اسکالر برمیگردوند داریم. در این جا D را یک Autoencoder می‌گذاریم. ورودی این Autoencoder یک تصویر است و خروجی آن نیز یک تصویر (تلاش برای بازسازی تصویر ورودی) است. تابع خطای بازسازی:

$$L(v) = |v - D(v)|^\eta$$

که در آن v ورودی (داده واقعی یا فیک) و η معمولاً برابر 1 (L1 Loss) یا 2 (L2 Loss) است.

یک Autoencoder که روی داده‌های واقعی آموزش دیده باشد، می‌تواند داده‌های واقعی را با خطای بسیار کمی بازسازی کند ($L(x)$ کوچک است). اما اگر یک داده فیک به آن بدهیم که قبلاً ندیده است، در بازسازی آن دچار مشکل شده و خطای بالایی خواهد داشت ($L(G(z))$ بزرگ است).

تابع هزینه متمایزگر و مولد به شکل زیر در می‌آید:

$$L_D = L(x) - k_t L(G(z))$$

$$L_G = L(G(z))$$

- تابع L_D : Autoencoder سعی می‌کند خطای بازسازی داده‌های واقعی ($L(x)$) را کمینه کند تا یاد بگیرد عکس‌های واقعی را خوب کدگذاری و دی‌کود کند. همزمان سعی می‌کند خطای بازسازی داده‌های فیک ($L(G(z))$) را بیشینه کند، یعنی عکس‌های فیک را خراب بازسازی کند.
- تابع L_G : Generator صرفاً سعی می‌کند خطای بازسازی داده‌های تولیدی خودش را کمینه کند.

حال این مدل یک مشکل اساسی مدل های قبلی را حل میکند. اونم تعادل قدرت بین G و D هست. گفتیم بالاتر که معمولا D سریع یاد میگیرد و قوی میشود و اجازه یادگیری G را نمیدهد. این مدل اما گفت تعادل زمانی رخ می دهد که امید ریاضی خطای بازسازی داده های واقعی با داده های فیک برابر شود و بخاطر همین یک هایپرپارامتر Diversity Ratio - γ تعریف کردند:

$$\gamma = \frac{E[L(G(z))]}{E[L(x)]}$$

که در آن $\gamma \in [0, 1]$.

- مقادیر پایین γ : به معنای تمرکز مدل روی کیفیت (کیفیت بازسازی تصویر بالاتر) اما تنوع کمتر است.
 - مقادیر بالای γ : به معنای تنوع بالاتر در تصاویر تولیدی است اما ممکن است کمی افت کیفیت به همراه داشته باشد.
- متغیر $k_t \in [0, 1]$ به عنوان یک ترم کنترل کننده معرفی می شود که تعیین می کند D چقدر باید روی جریمه کردن تصاویر فیک تمرکز کند. این متغیر در هر قدم آپدیت می شود:

$$k_{t+1} = k_t + \lambda_k(\gamma L(x) - L(G(z)))$$

در این فرمول λ_k نرخ یادگیری برای k است.

- اگر $L(G(z))$ بیش از حد افت کند (یعنی G خیلی قوی شده و D را گول زده)، مقدار عبارت داخل پرانتز مثبت می شود، k_t افزایش می یابد و D در قدم بعدی تمرکز بیشتری روی تشخیص فیک ها می گذارد.
- اگر D خیلی قوی شود، k_t کاهش می یابد و تمرکز D از روی G برداشته میشه.

$$L_D = L(x) - k_t L(G(z))$$

یعنی میزان اهمیت D به G را یک جوری به صورت dynamically میتونیم کنترل کنیم تا آموزش راحت تر بشه.

یک معیار همگرایی هم تعریف کرد که هرچه این عدد به صفر نزدیک تر شود، یعنی مدل به تعادل و همگرایی رسیده و کیفیت عکس ها بهتر است

$$\mathcal{M}_{global} = L(x) + |\gamma L(x) - L(G(z))|$$

یعنی می خواهیم هم خطای بازسازی واقعی ($L(x)$) کم شود و هم فاصله خطای فیک و واقعی به نسبت γ صفر شود.

خلاصه که استفاده از Autoencoder باعث میشود بجای feature های local بتواند feature های global را پیدا کند (این چیز مهمیه چون در GAN ها اگر حواسمون به این نباشد G برای گول زدن D مثلا میتواند در حد یک فیلتر رنگی عمل کند و D هم راحت گول میخورد ولی ما منطقا این مولد

بدرموم نمیخوره و یک چیزی میخوایم که خیلی با کیفیت تر و وضوح بیشتر عکس ها را تولید کند و بخاطر همین باید مجبورش کنیم که نگاهش را global تر کند). در Naive-GAN که مکانیسمی برای پایداری نداشت به اون صورت و WGAN-GP هم با جریمه کردن گرادینان سعی میکند کنترل کند و در اینجا به صورت dynamic سعی میکند D و G قدرتشون در حد هم بموند و آموزش پایدارتر بشود. M_{global} هم به ما کمک میکند که در طول آموزش ببینیم آیا دارد کاهش میابد یا نه و چون به معنای کیفیت عکس ها هست دیگر نیاز نیست مثلاً یک نمونه برداری کنیم در هر اپاک و چشمی بررسی کنیم.

۴.۱.۱ معیار ارزیابی FID

برای اینکه چشمی دیگر کیفیت تصاویر را نسنجیم که کار سخته معیار FID تعریف شد. اینکه ما بیایم دو مجموعه عکس را پیکسل پیکسل مقایسه کنیم کار اشتباهیه چون مثلاً shift invariant نمیشود و یک شیفت مکانی کوچک باعث میشه این عدد خیلی زیاد بشود. معیار FID به جای پیکسل ها، ویژگی های سطح بالا را مقایسه می کند. برای این کار، از یک شبکه از پیش آموزش دیده به نام Inception-v3 (که روی مجموعه داده ImageNet آموزش دیده است) استفاده می شود.

- تصاویر واقعی و تصاویر تولید شده (فیک) به این شبکه داده می شوند.
- لایه آخر طبقه بندی (Softmax) حذف شده و خروجی آخرین لایه Pooling (یک بردار با طول 2048 برای هر تصویر) به عنوان نمایشی از ویژگی های معنایی تصویر استخراج می شود.
- در فضای ویژگی های 2048-بعدی، FID فرض می کند که هم ویژگی های عکس های واقعی و هم ویژگی های عکس های فیک، از یک توزیع گاوسی چندمتغیره پیروی می کنند.

۱. میانگین (μ): یک بردار 2048-بعدی که مرکز توزیع را نشان می دهد.
۲. ماتریس کوواریانس (Σ): یک ماتریس 2048×2048 که میزان پراکندگی و تنوع داده ها را در این فضا نشان می دهد.

پس ما دو مجموعه پارامتر داریم:

- برای داده های واقعی: (μ_r, Σ_r)
- برای داده های فیک: (μ_g, Σ_g)

$$FID = ||\mu_r - \mu_g||_2^2 + Tr\left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\right)$$

به صورت شهودی یعنی وقتی میخوایم دو توزیع گاوسی را بررسی کنیم چقدر فاصله دارند. اول به ذهنمون میرسد که فاصله میانگین های آنها را اندازه بگیریم که میشود ترم اول (ایا کیفیت کلی تصویر های فیک اندازه واقعی هست یا نه) و بعد هم میگیریم فاصله بین ماتریس های کوواریانس آنها را محاسبه کنیم یعنی میزان پراکندگی و تنوع دو توزیع را بررسی کنیم (آیا تنوع عکس های فیک به اندازه تنوع عکس های واقعی هست یا نه)

خوبی این معیار مثلاً نسبت به IS این هست که نسبت به Mode collapse حساس هست (چون اگر تنوع نداشته باشد ماتریس کوواریانس فیک (Σ_g) کوچک می شود و اختلاف آن با ماتریس کوواریانس واقعی (Σ_r) در ترم دوم فرمول به شدت افزایش می یابد) و مستقیماً این معیار کیفیت تنوع را با Ground Truth مقایسه میکند. FID یک فاصله است و یعنی هر چه کمتر باشد مدل مولد ما بهتر هست.

۲.۱ پیاده سازی

۱.۲.۱ مدل Naive GAN

1. آماده سازی داده ها

در این پیاده سازی از مجموعه داده استاندارد MNIST استفاده شده است. لایه آخر مولد Tanh استفاده کردیم پس دیتا هم باید اسکیل کنیم تا مشکلی نداشته باشد در محاسبتان، داده ها پس از تبدیل شدن به تنسور، با استفاده از ترنسفورم $\text{Normalize}(\text{mean}=(0.5,), \text{std}=(0.5,))$ به بازه $[-1, 1]$ نگاشت شده اند. اندازه مینی بچ ها برابر با 64 تنظیم شده است.

2. معماری شبکه ها

مدل پیاده سازی شده در عمل از معماری DCGAN (Deep Convolutional GAN) برای مدل پایه بجای MLP ساده استفاده کردم.

شبکه مولد (G - Generator):

- ورودی: بردار نویز تصادفی z با ابعاد 100 از توزیع نرمال استاندارد.
- از 4 لایه کانولوشن ConvTranspose2d برای افزایش تدریجی ابعاد مکانی (Upsampling) استفاده شده است.
- استفاده از BatchNorm2d پس از هر لایه (به جز لایه آخر) برای پایدارسازی آموزش، و استفاده از تابع فعال ساز ReLU. در لایه خروجی برای تولید تصویر نهایی در بازه مجاز، از تابع Tanh استفاده شده است.

شبکه متمایزگر (D - Discriminator):

- یک شبکه عصبی CNN شامل 4 لایه کانولوشن با Stride برابر با 2 جهت کاهش ابعاد مکانی (Downsampling).
- استفاده از تابع فعال ساز LeakyReLU با شیب منفی 0.2 که به جریان بهتر گرادیان ها کمک می کند. لایه آخر از تابع Sigmoid برای فشرده سازی خروجی در بازه $[0, 1]$ (جهت محاسبه احتمال واقعی بودن تصویر) استفاده کردم.

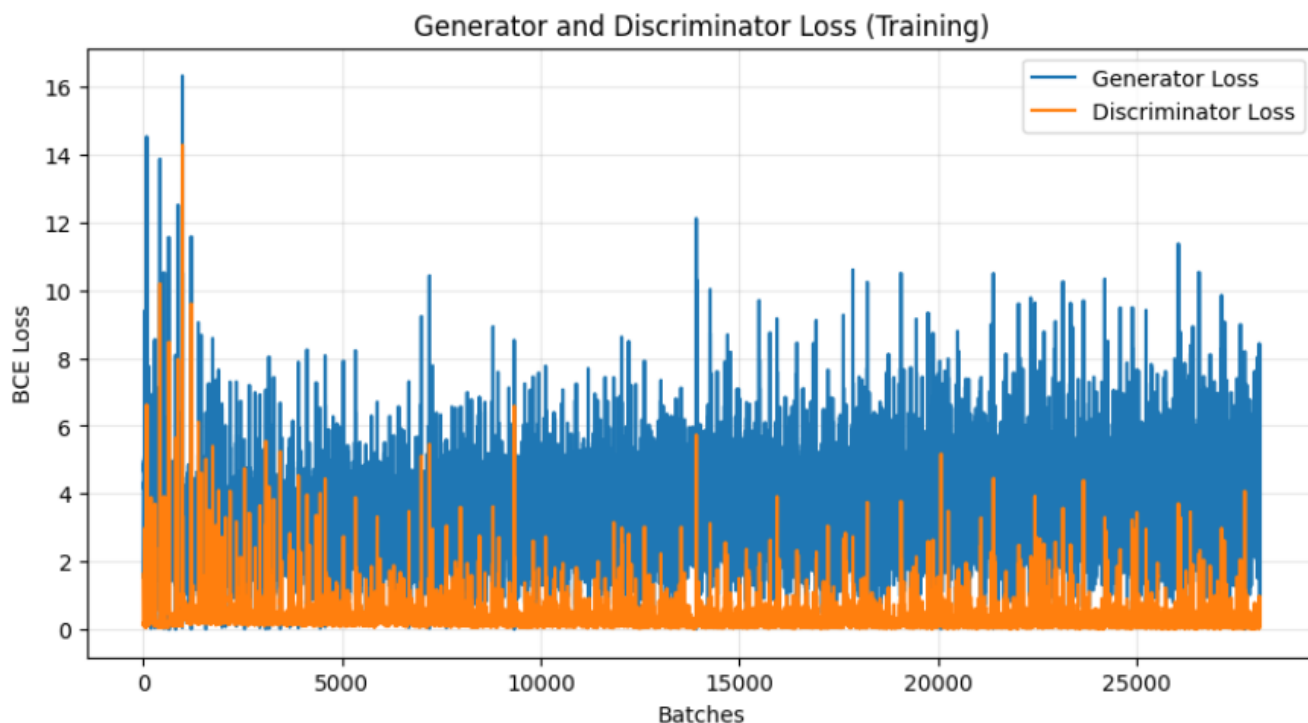
وزن های لایه های کانولوشن از یک توزیع نرمال $\mathcal{N}(0, 0.02)$ و وزن های لایه های BatchNorm از توزیع $\mathcal{N}(1.0, 0.02)$ به صورت دستی مقداردهی اولیه شده اند تا از تقارن جلوگیری شده (همه وزن های فیلتر ها در یک لایه خاص به یک اندازه و جهت آپدیت نشود و با شکوندن تقارن میتونیم ظرفیت یادگیری را بیشتر کنیم.) و آموزش سریع تر همگرا شود.

3. تنظیمات آموزش

- از تابع خطای nn.BCELoss استفاده شده است.
- برای هر دو شبکه از الگوریتم Adam با نرخ یادگیری $\alpha = 0.0002$ استفاده گردید. پارامتر مومنتوم اول $(\beta_1 = 0.5)$ تنظیم شده است تا نوسانات آموزش کاهش یابد.
- از ترفند Non-saturating Loss برای شبکه مولد استفاده شده است؛ به این معنا که شبکه G به جای کمینه کردن خطای تشخیص فیک بودن عکس هایش، تلاش می کند خطای واقعی تشخیص داده شدن آن ها $(\text{real_label} = 1.0)$ را توسط متمایزگر کمینه کند.

- مدل برای 30 اپیاک آموزش داده شده است.

4. نتایج و تحلیل ارزیابی‌ها



شکل ۱.۱: نمودار خام تابع هزینه مولد و متمایزگر در طول آموزش (به ازای هر Batch)

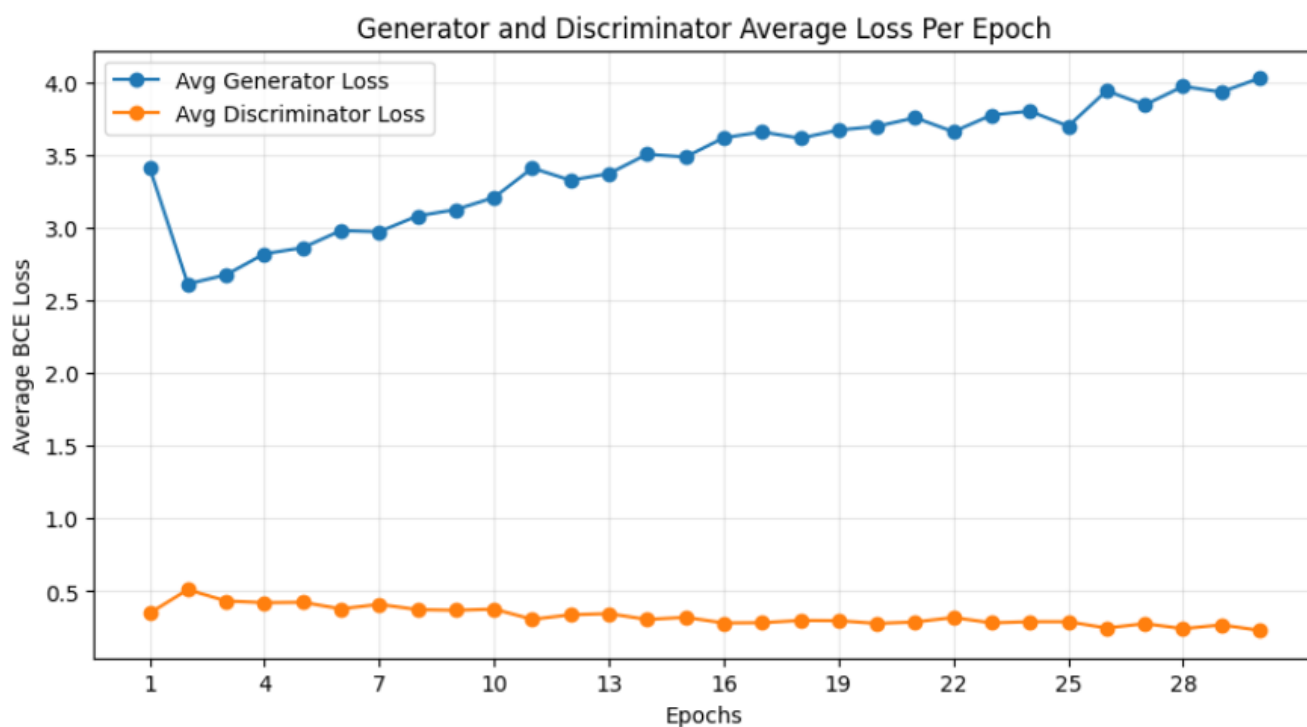
1- 4 تحلیل نمودارهای تابع هزینه (Loss Analysis) تحلیل نمودارها:

- در نمودار میانگین اپیاک‌ها مشاهده می‌شود که خطای D (خط نارنجی) از همان ابتدا افت کرده و در مقادیر بسیار پایین (حدود 0.3) ثابت می‌ماند. این یعنی D به سرعت یاد گرفته است که داده‌های واقعی را از فیک تشخیص دهد. همزمان با قوی شدن D، خطای شبکه G (خط آبی) به جای کاهش، روندی صعودی به خود گرفته و از حدود 2.5 به بالای 4.0 می‌رسد.
- در بخش تئوری گفتیم که در ترفند Non-saturating Loss که برای جلوگیری از vanishing gradient می‌زدیم یک بدی دارد که باعث می‌شود واریانس گرادینت‌ها خیلی زیاد بشود که با دیدن نمودار به ازای هر Batch این به صورت عملی هم دیده می‌شود.

2- 4 تحلیل کیفی تصاویر تولید شده تحلیل کیفی:

با وجود نوسانات تابع هزینه، شبکه توانسته است ساختار کلی ارقام (مانند 4، 6، 0 و 7) را تا حد قابل قبولی یاد بگیرد. با این حال کمی Distortion نیز دیده می‌شود و پرفکت نیست.

- 3- 4 ارزیابی کمی با معیار FID برای سنجش دقیق کیفیت تصاویر تولید شده نسبت به توزیع داده‌های واقعی MNIST، معیار FID بر روی نمونه‌های نهایی محاسبه گردید.



شکل ۲.۱: نمودار میانگین تابع هزینه مولد و متمایزگر به ازای هر Epoch

• نمره نهایی FID کسب شده: 5.9782

۲.۲.۱ مدل WGAN-GP

1. معماری

شبکه مولد (Generator):

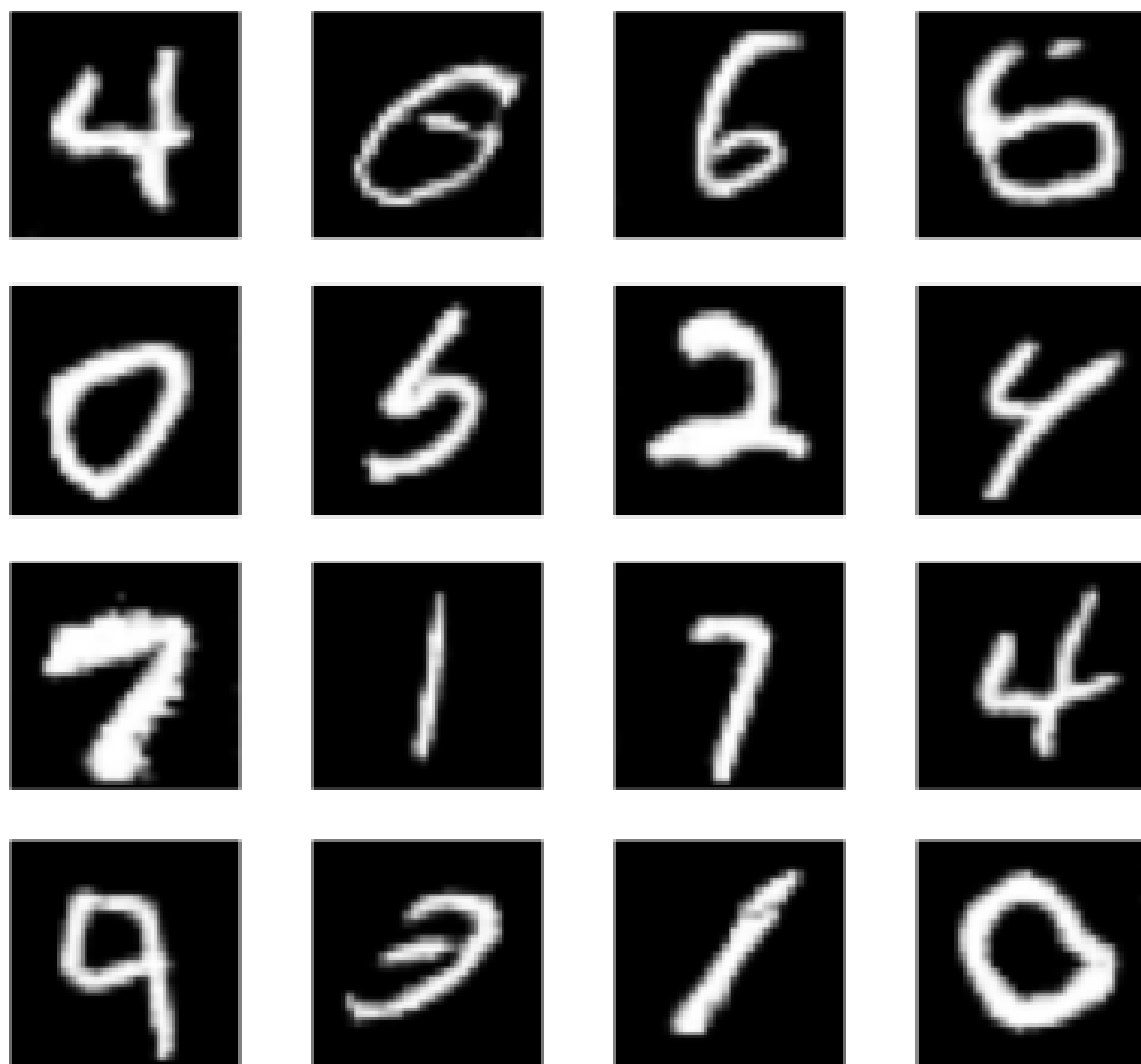
ساختار شبکه مولد نسبت به مدل Naive تغییر نکرده است.

شبکه منتقد (Critic):

• در لایه آخر شبکه Critic، تابع Sigmoid حذف شده است. خروجی این شبکه یک اسکالر واقعی (تخمینگر فاصله) است که محدود به بازه $[0, 1]$ نیست.

• برای اعمال صحیح Gradient Penalty (که نیازمند محاسبه گرادیانهای مستقل برای هر نمونه ورودی است)، لایههای BatchNorm2d حذف شده و با LayerNorm جایگزین شد.

Naive GAN - Final Results



شکل ۳.۱: نمونه ارقام تولید شده توسط شبکه مولد در پایان ایپاک 30

2. تنظیمات آموزش و تابع هزینه

• حلقه آموزش نامتقارن: برای اینکه شبکه منتقد بتواند تقریب خوبی از فاصله واسراشتاین ارائه دهد، به ازای هر 1 بار آپدیت وزنهای Generator، شبکه Critic تعداد $n_{critic} = 2$ بار آپدیت می‌شود. در مقاله این عدد 5 گفته شده بود ولی چون باید 30 ایپاک را ثابت نگه میداشتم تا مقایسه عادلانه با Naive میشد. اون وقت 6 بار بیشتر G آپدیت نمیشد که منطقاً نتایج خوبی نمیداد نسبت به Naive ای که 30 بار دارد آپدیت میشود. 2 را انتخاب کردم و آموزش دادم و همینطور که از FID پایین تر میبینیم خیلی نزدیک به Naive نزدیک شده است (با اینکه در مقایسه عادلانه شاید باید 2 برابر بیشتر آموزش میدادم این مدل را).

• تابع هزینه Critic: تابع هزینه این شبکه از سه بخش تشکیل شده است:

۱. بیشینه کردن نمره داده‌های واقعی (یا کمینه کردن منفی آن: $-\text{avg_real_score}$)

۲. کمینه کردن نمره داده‌های فیک ($+\text{avg_fake_score}$)

۳. اعمال ترم جریمه گرادیان ($\text{lambda_gp} * \text{gp}$) با ضریب $\lambda = 10$.

• تابع هزینه Generator: صرفاً در جهت بیشینه کردن نمره داده‌های تولیدی توسط شبکه منتقد ($-\text{torch.mean}(c_fake_score)$) تعریف شده است.

• محاسبه Gradient Penalty: مطابق با پیاده‌سازی، نمونه‌های interpolation ($\hat{x}$) با استفاده از linear interpolation (با یک متغیر تصادفی real_prob بین 0 و 1) بین تصاویر واقعی و تصاویر فیک تولید شده در همان گام، ایجاد می‌شوند. سپس با استفاده از torch.autograd ، گرادیان خروجی شبکه منتقد نسبت به $\hat{x}$ محاسبه شده و جریمه انحراف از نرم-2 برابر با 1 (شرط 1-لیپشیتز) بر روی آن اعمال می‌گردد.

3. نتایج و تحلیل ارزیابی‌ها

1- 3 تحلیل نمودارهای تابع هزینه تحلیل نمودارها:

• برخلاف نوسانات شدید و اسپایک‌های متوالی در Naive GAN (که ناشی از انفجار گرادیان بود)، در اینجا نمودار Loss بسیار هموارتر و نرم‌تر رفتار می‌کند.

• نمودار به ازای هر ایپاک هم که نگاه میکنیم میبینیم خیلی نرم تر هست و به همگرایی رسیده و توانستیم عملاً اون ذات Minimax که خیلی آموزشش سخت است و تابع هزینه‌ها معمولاً خیلی بالا پایین میشوند را اینجا خیلی خوب کنترل کنیم.

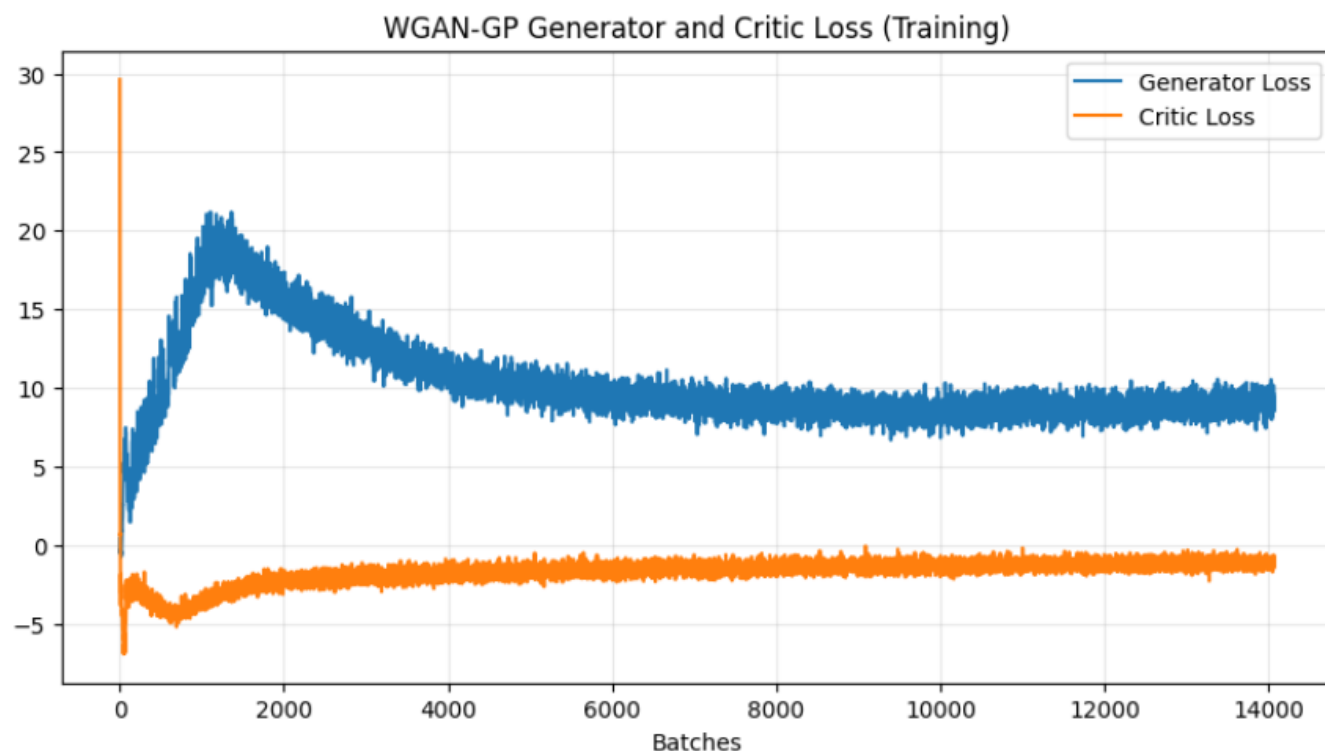
2- 3 تصاویر تولید شده

3- 3 ارزیابی کمی با معیار FID

• نمره نهایی FID کسب شده: 9.5857

تحلیل FID:

نمره FID کسب شده در WGAN-GP (حدود 9.5) نسبت به Naive GAN (حدود 5.9) بالاتر شده است ولی گفتیم این به این معنا نیست که WGAN-GP از Naive بدتر عمل کرده است بلکه واقعاً خیلی خوب بوده است به ازای کمتر نصف آپدیت برای شبکه مولد و اگر دستانمان باز بود که ایپاک



شکل ۴.۱: نمودار خام تابع هزینه مولد و منتقد در طول آموزش WGAN-GP (به ازای هر Batch)

را بیشتر کنیم آن موقع احتمالاً FID اش از Naive هم خیلی کمتر میشد.

۳.۲.۱ مدل BEGAN

1. معماری

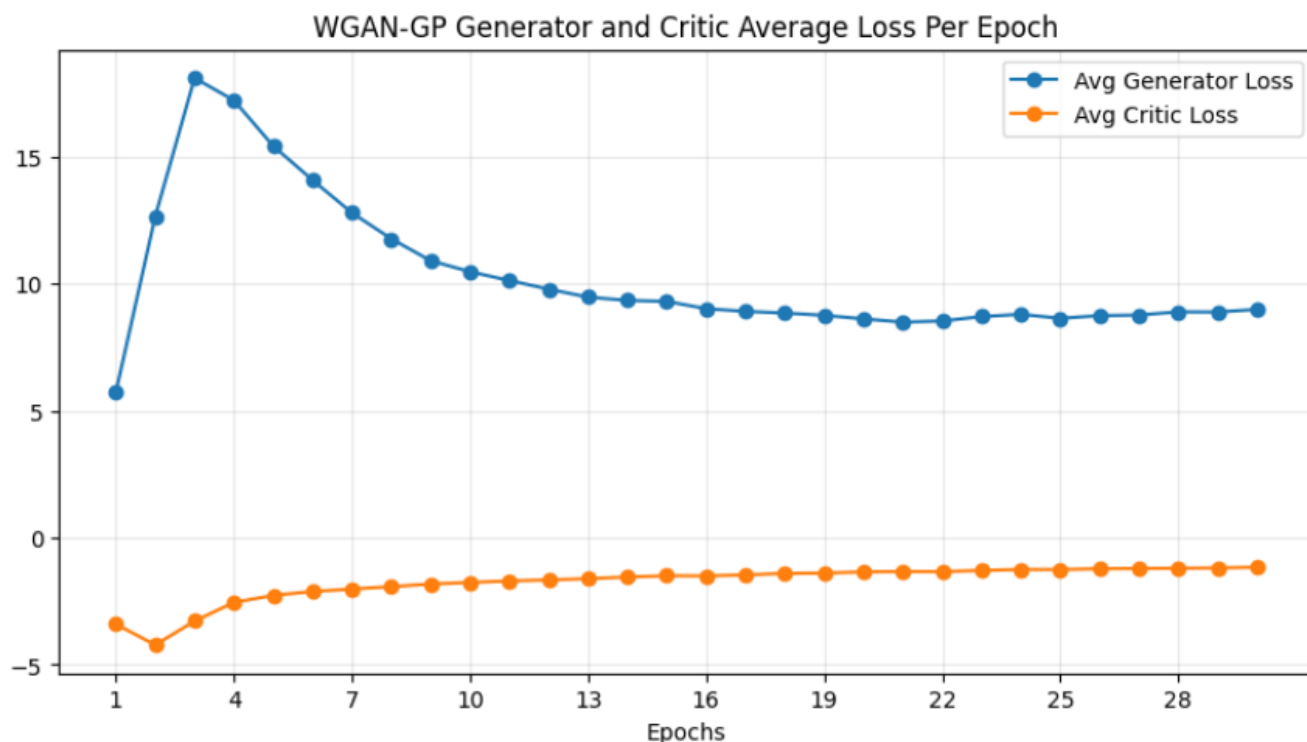
شبکه مولد (Generator):

ساختار شبکه مولد نسبت به مدل Naive تغییر نکرده است.

شبکه متمایزگر (Discriminator - D):

- ساختار Autoencoder: در Encoder با لایه‌های کانولوشن (stride 2)، تصویر ورودی به یک فضای پنهان با ابعاد $\text{latent_dim} = 256$ نگاشت میکنیم. در Decoder هم که برعکسش را با کمک ConvTranspose2d انجام میدیم.
- از تابع فعال‌ساز نمایی خطی (nn.ELU) استفاده شده است که به نرم‌تر شدن فضای بازسازی کمک می‌کند.

همانطور که در جلوتر خواهیم دید این مدل نتایج خوبی نداد و یکی از ایده‌هایی که داشتیم این بود که وزن‌های لایه‌های کانولوشن با استفاده از روش Kaiming Normal مقداردهی شده‌اند که جلوی محو شدن گرادیان را بگیرد.



شکل ۵.۱: نمودار میانگین تابع هزینه مولد و منتقد به ازای هر Epoch (WGAN-GP)

2. تنظیمات آموزش

- تابع خطای پایه: به جای BCE یا واسراشتاین، از خطای بازسازی پیکسل به پیکسل با نرم L_1 (nn.L1Loss) استفاده شده است.
- هایپر پارامترهای کنترلی:

– نسبت تنوع $\gamma = 0.7$ تنظیم شده است.

– نرخ یادگیری متغیر کنترل $\lambda_k = 0.001$ می باشد.

– نرخ یادگیری شبکه ها بسیار کوچک تر از قبل و برابر با $\alpha = 3.5e - 5$ با بهینه ساز Adam تعیین شده است.

– پس از آپدیت وزن های G و D، متغیر کنترلی k_t در هر قدم (Step) با فرمول کنترل کننده تناسبی آپدیت شده و در بازه $\text{Clamp}[0.01, 1.0]$ می شود (از قصد نیز از 0 بشه چون اون موقع D یک Autoencoder میشود عملا و دیگر اهمیتی به عکس فیک نمیده پس G هم دیگه بهتر نمیشود و عملا دیگه همونجا یادگیریش متوقف میشود که در ایپاک های حدود 3 این اتفاق میوفتاد وقتی 0 میگذاشتیم این خط کد را و نتایج عملا حتی نمیشد فهمید MNIST هستند).

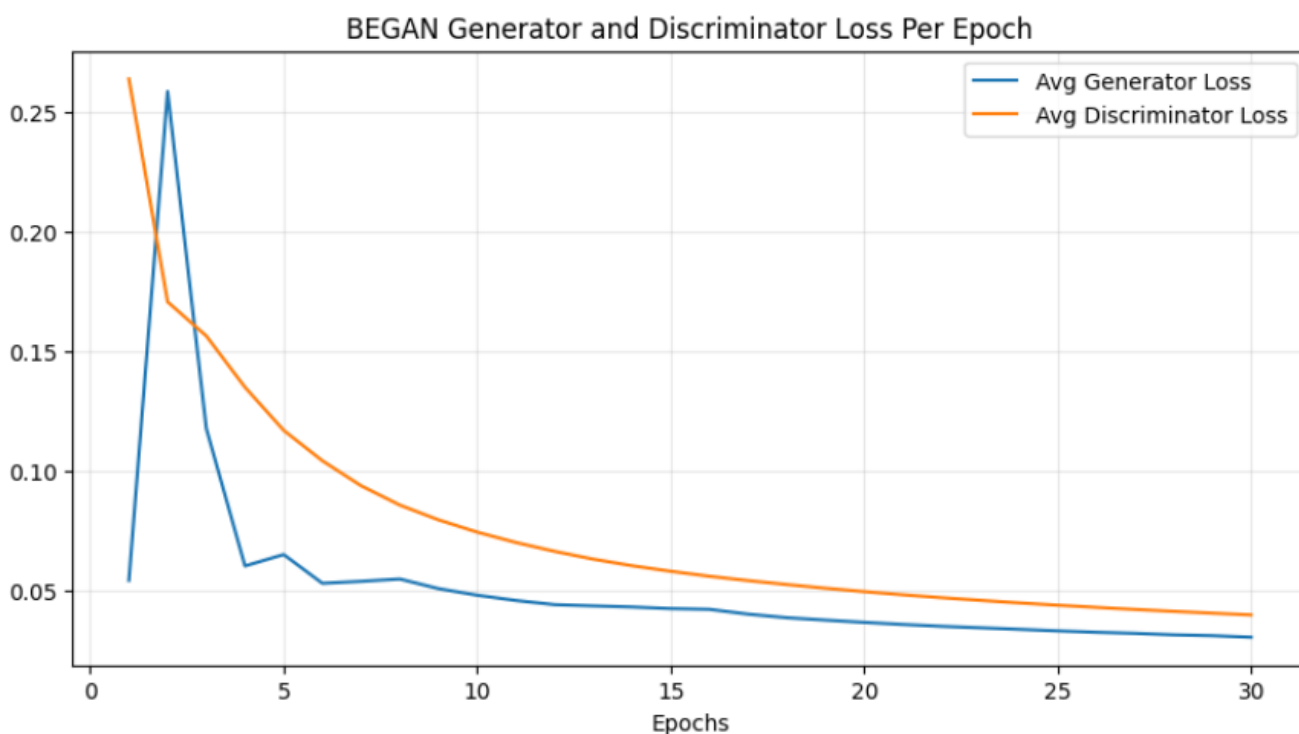
3. نتایج و تحلیل ارزیابی ها

1- 3 تحلیل نمودارهای تابع هزینه و همگرایی سراسری در نمودار اول، بر خلاف Naive GAN که دچار نوسان بود، شاهد یک روند نزولی بسیار پایدار برای هر دو شبکه هستیم. خطای بازسازی (L_1) به سرعت کاهش یافته و در مقادیر بسیار پایینی ثابت شده است که نشان می دهد متمایزگر توانسته در

WGAN-GP GAN - Final Results



شکل ۶.۱: نمونه ارقام تولید شده توسط شبکه مولد WGAN-GP در پایان ایپاک 30



شکل ۷.۱: نمودار میانگین تابع خطای بازسازی مولد و متمایزگر به ازای هر Epoch (BEGAN)

نقش خود به عنوان یک Autoencoder به خوبی عمل کند.

در نمودار دوم:

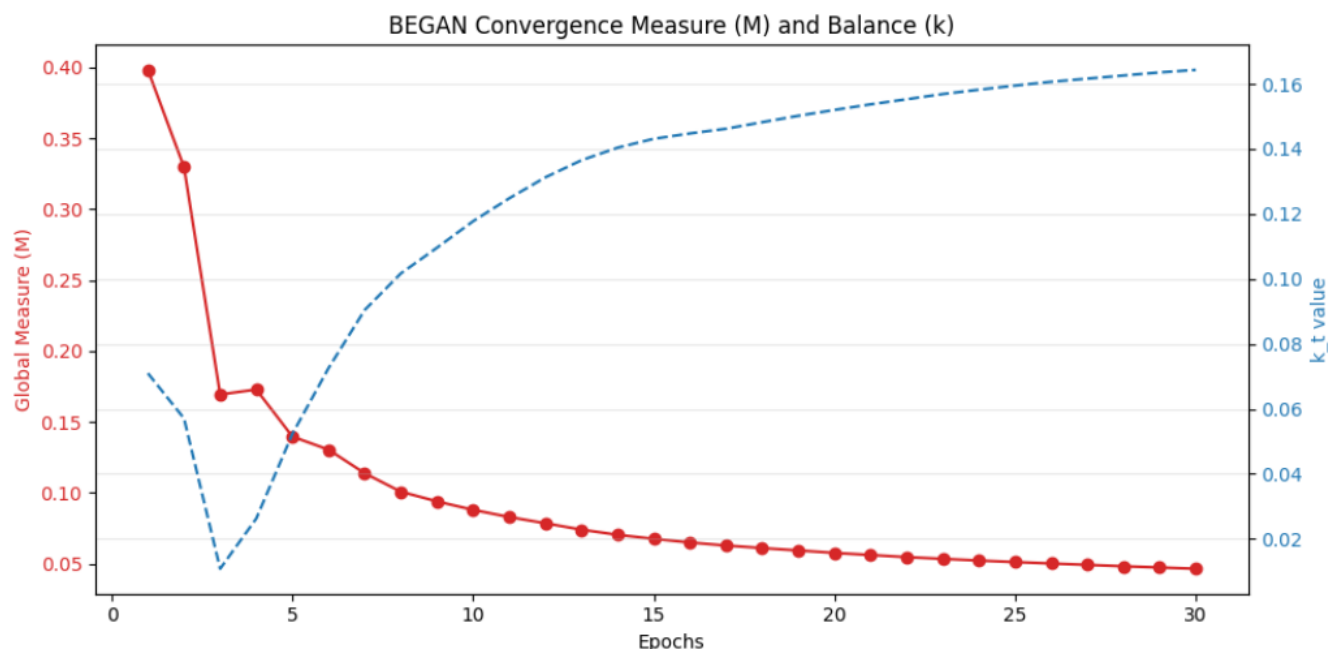
- معیار همگرایی سراسری ($\mathcal{M}$): خط قرمز رنگ (M) به طور پیوسته و با یک شیب نرم در حال کاهش است و به سمت صفر میل می‌کند. این یعنی شبکه از لحاظ تئوری به تعادل مطلوب همگرا شده است.
- متغیر کنترل (k_t): خط چین آبی رنگ نشان می‌دهد که مقدار k_t از صفر شروع شده و با گذشت زمان و قوی‌تر شدن مولد، برای حفظ تعادل مقدار آن تا حدود 0.16 افزایش یافته است.

2-3 تحلیل کیفی تصاویر تولید شده تحلیل کیفی:

اعداد خیلی واضح و شارپ و با کیفیت هستند ولی دچار Mode Collapse شدید شده است.

یک سری از احتمالاتی که می‌دهم که شاید بخاطر اینا mode collapse رخ داده:

- در معماری شبکه D، بعد فضای پنهان برابر با $\text{latent_dim} = 256$ تنظیم شده است ولی MNIST دارای ابعاد 28×28 (معادل 784 پیکسل) هستند، نسبت فشرده‌سازی در این Autoencoder تقریباً 3 به 1 است. در نتیجه، D به جای یادگیری ویژگی‌های سطح بالا، به راحتی تمام داده‌های واقعی را حفظ می‌کند و خطای بازسازی $L(x)$ با سرعت بسیار بالایی به سمت صفر میل می‌کند.



شکل ۸.۱: نمودار معیار همگرایی سراسری (M) و متغیر کنترل تعادل (k) در طول آموزش

• در این پیاده سازی، نسبت تنوع برابر با $\gamma = 0.7$ در نظر گرفته شده است. طبق معادله تعادل BEGAN، سیستم تلاش می کند به نقطه $E[L(G(z))]$ برسد. به دلیل ظرفیت بالای D ، $L(x)$ بسیار کوچک می شود. حال با اعمال ضریب 0.7، شبکه G تحت فشار شدیدی قرار می گیرد تا تصاویری تولید کند که خطای بازسازی آن ها حتی از تصاویر واقعی هم کمتر باشد. تنها راهی که G برای ارضای این شرط سخت گیرانه پیدا می کند، توقف تولید ارقام پیچیده با خطوط منحنی (مثل 8 یا 3) و تولید مکرر ساده ترین اشکال هندسی (مانند عدد 1) است که کمترین ریسک خطای پیکسل به پیکسل را در Autoencoder دارند.

• شبکه G صرفاً تلاش می کند خطای L_1 تصاویر خود را در خروجی Autoencoder کمینه کند ($L_G = L_1(D(G(z)), G(z))$). ارقامی که دارای مساحت پیکسلی کمتر و ساختار خطی هستند (مثل 1 و 7)، در صورت جابجایی جزئی توسط Autoencoder، خطای L_1 بسیار کمتری نسبت به ارقام پیچیده تر و دارای حلقه (مثل 0، 8 و 6) تولید می کنند. از آنجا که BEGAN برخلاف WGAN به طور صریح فاصله بین دو توزیع کلان را جریمه نمی کند و صرفاً به کیفیت بازسازی اهمیت می دهد، Generator ترجیح می دهد تنوع را به طور کامل قربانی کند تا کیفیت بازسازی این ارقام ساده را در حد کمال نگه دارد.

3-3 ارزیابی کمی با معیار FID

• نمره نهایی FID کسب شده: 58.0975

همان طور که در بخش تئوری FID اشاره شد، این معیار به تنوع تصاویر تولیدی حساسیت بسیار بالایی دارد. از آنجا که مدل BEGAN شما دچار Mode Collapse شده و فقط چند رقم خاص را تولید می کند، ماتریس کوواریانس توزیع تولیدی آن بسیار کوچک شده و فاصله آن با ماتریس کوواریانس توزیع واقعی کل اعداد (که شامل همه 10 رقم است) به شدت افزایش یافته است.

BEGAN Final Output



شکل ۹.۱: نمونه ارقام تولید شده توسط شبکه مولد BEGAN در پایان ایپاک 30

۴.۲.۱ مقایسه مدل های GAN

در این بخش، عملکرد سه معماری پیاده سازی شده (Naive GAN، WGAN-GP و BEGAN) مقایسه میشود.

1. مقایسه از نظر پایداری آموزش و نمودارهای خطا

• مدل Naive GAN:

نمودار خطای این مدل نشان دهنده نوسانات شدید (Spikes) و ناپایداری گرادیان است. متمایزگر به سرعت بر بازی مسلط شده و خطای آن در مقادیر پایین تثبیت می شود، در حالی که خطای شبکه مولد روندی صعودی و پرنوسان دارد. در این مدل، عدد Loss هیچ اطلاعات معناداری از کیفیت تصاویر به ما نمی دهد و صرفاً نشان دهنده یک رقابت نامتعادل است. (خروجی های در طول آموزش اگر نگاه شود. مدل دارد بهتر کیفی عمل میکند ولی واقعا ربطی به loss ندارد و نمیتوان اطلاعات معناداری از رو اون پیدا کرد)

• مدل WGAN-GP:

جایگزینی فاصله JSD با فاصله واسراشتاین و استفاده از Gradient Penalty، نوسانات را به طور کامل از بین برده است. نمودار خطای شبکه منتقد در این مدل به یک مقدار منفی ثابت همگرا می شود که در واقع تخمینی معنادار از فاصله واسراشتاین بین دو توزیع است. در این مدل، کاهش خطای منتقد مستقیماً با بهبود کیفیت تصاویر همبستگی دارد و پایداری آموزش تضمین شده است. (میتوانید به خروجی های در طول آموزش نگاه کنید که هم فاصله خروجی گرفته شده است و هم به صورت کیفی میتوانید در ایپاک های متفاوت مشاهده کنید و تاییدیه حرف اینجام هست)

• مدل BEGAN:

این مدل با استفاده از تئوری کنترل (متغیر k_t)، نرم ترین و هموارترین نمودار همگرایی را ارائه می دهد. خطای L1 برای هر دو شبکه به سرعت کاهش یافته و به سمت صفر میل می کند. همچنین معیار همگرایی سراسری (M_{global}) به شکلی پیوسته کاهش می یابد.

2. مقایسه از نظر کیفیت بصری و تنوع

خروجی ها بعد از 30 ایپاک بررسی شدند.

• مدل Naive GAN و WGAN-GP:

تصاویر تولید شده کیفیت و تنوع بالایی دارند و در بخش WGAN-GP هم مقایسه کیفی اش را با Naive توضیح دادم که در مقایسه کامل عادلانه بهتر خواهد عمل کرد ولی اینجا چون تعداد ایپاک را باید ثابت نگه داریم باز خیلی تونسته نزدیک بشه.

• مدل BEGAN:

ارقام تولید شده به دلیل استفاده از Autoencoder و خطای L1 شارپ ترین و واضح ترین خروجی ها در بین هر سه مدل هستن ولی مدل دچار Mode Collapse بسیار شدید شده است و به جای تولید تمام ارقام، صرفاً به تولید ارقام ساده ی خطی (مانند 1 و 7 و 9) بسنده کرده است تا خطای بازسازی خود را کمینه نگه دارد. (توضیح چرایی اش در بخش BEGAN گفته شد.)

3. مقایسه از منظر ارزیابی کمی (معیار FID)

جدول زیر نمرات نهایی FID برای هر سه مدل را نشان می‌دهد:

وضعیت کلی	FID	مدل پیاده‌سازی شده
خوب	5.9782	Naive GAN
متوسط رو به خوب	9.5857	WGAN-GP
بسیار ضعیف	58.0975	BEGAN

با وجود ناپایداری نمودارها، Naive GAN در ایپاک 30 توانسته است تعادل قابل قبولی بین کیفیت و تنوع ایجاد کند. از آنجا که تمام 10 رقم در خروجی وجود دارند، ماتریس کوواریانس توزیع تولیدی به خوبی با توزیع واقعی منطبق شده و نمره 5.97 را ثبت کرده است. با وجود کیفیت بصری بالاتر و پایداری عالی، WGAN-GP نمره 9.58 را کسب کرده است. دلیل این امر سرعت همگرایی کندتر این معماری است. به دلیل آپدیت‌های کمتر شبکه مولد (نسبت 1 به 2 به 2 منتقد) و محدودیت‌های سخت‌گیرانه Gradient Penalty و این مدل برای نتایج خیلی بهتر نیاز دارد نسبت به Naive تعداد ایپاک بیشتری داشته باشد. BEGAN اما خیلی FID اش بدتر شده است، با وجود تولید عکس‌های بسیار شارپ، یک کلاس درس عملی برای درک معیار FID است. معیار FID به شدت به تنوع داده‌ها (ماتریس کوواریانس توزیع) حساس است. از آنجا که BEGAN دچار Mode Collapse شدید شده و فقط ارقام 1 و 7 را تولید کرده است، فاصله توزیع آن با توزیع واقعی (که شامل 10 رقم متنوع است) به شدت افزایش یافته و سیستم جریمه سنگینی را در قالب یک نمره FID بالا به مدل اعمال کرده است.

در نهایت با توجه به نتایج من مدل Naive برای وقتی که می‌خواهیم آموزش سریع تموم بشود نقطه شروع خوبی است اما اگر می‌توانیم تعداد ایپاک بیشتر آموزش بدهیم WGAN-GP بهترین نتیجه را می‌دهد از کیفیت و تنوع.